

PMOS -- Perpetual Multi-Agent Orchestration System

The living graph is the central data structure of PMOS. Every intermediate LLM response that spawns sub-questions becomes a new TaskNode in Neo4j before execution proceeds. There are no static DAGs -- the graph is always live.

Neo4j stores TaskNode, ExecutionEvent, SOPNode, and AgentCapabilityNode entities connected by relationships such as SPAWNED_BY, ASSIGNED_TO, FOLLOWS_SOP, BELONGS_TO, and UPDATED_FROM.

MySQL handles relational data: agents, tools, teams, scoring weights, RL feedback logs, score history, memory tables, capability registry, and execution graph logs. Each service owns its own set of tables and never writes to another service's tables.

Redis provides pub/sub and stream-based communication between services. Key streams include orchestrator:tasks, memory:writes, scoring:feedback, events:telemetry, events:tool_health, events:capability_added, and events:documents.

Qdrant (or FAISS / Pinecone / Weaviate via config) powers the vector search backend for the RAG pipeline. The RAG service performs parallel retrieval from up to four sources, then re-ranks candidates with a cross-encoder model.

Scoring uses six weighted factors: Relevance (R), Accuracy (Acc), Tool Success (P), Latency Penalty (Lat), Memory Utilization (Conf), and Validation Pass (Know). Weights are adapted via reinforcement learning -- the system never uses hard-coded thresholds.

Adaptive bands are computed from rolling execution history using:

$$\text{band} = \text{rolling_mean}(\text{last_N}) \pm (\text{rolling_std}(\text{last_N}) * \text{sensitivity_factor})$$
with minimum width enforcement and absolute floor/ceiling.

The memory subsystem has four tiers:

1. SHORT_TERM -- Redis hash with TTL
2. LONG_TERM -- MySQL + per-agent FAISS shard
3. REASONING -- MySQL JSON column, updated via distillation
4. EPISODIC -- MySQL table with full episode blobs + embeddings

Every system prompt is assembled at runtime from all four memory tiers -- no prompt literals appear in source code.

The meta-assembly service detects capability gaps and generates new tool, skill, or agent specifications using an LLM. Generated code runs in an isolated subprocess with timeout and resource limits. Safety checks prohibit system calls, file writes outside sandbox, and network calls to non-whitelisted hosts.